Otto: AI prompts

This document has two parts.

1. **The prompts inside Otto:** everything Otto sends to the model, including the base instructions, the personas and the guardrails. These are the real prompts from `web/src/lib/ai/`, shown lightly condensed. Words in `{curly brackets}` are filled in at run time.
2. **How we used AI to build Otto:** the prompts that shaped the project while we built it with Claude Code.

Part 1. The prompts inside Otto

How a request to the model is built

- **Model:** Claude (Anthropic API), set in one place (`web/src/lib/config.ts`).
- **System prompt:** the same for every call. It sets Otto's persona and contains the safety rules (both below).
- **Task prompt:** each feature sends one short message describing the job, with the context it needs: the task, the current step, and sometimes one photo from the camera.
- **Structured answers:** every answer must match a strict JSON schema. The descriptions of the fields are part of the instructions (for example "12 words or fewer"). The server rejects anything that doesn't match.
- **Guardrails in code, not only in words:** the most important safety behaviours are enforced by the server whatever the model writes (see "Guardrails" below).
- **Context we never send:** no full names (first names only), no contact details, no other students' data. Photos are used for one answer and never stored.

Base instructions and persona (system prompt, every call)

```
You are Otto, a calm step-by-step coach for autistic high school students and young
adults up to 22 learning job tasks at a school work site (a cafe, a shirt studio, a
shipping store).
Use short, plain, literal words. One idea per sentence. No idioms or sarcasm.
Be kind and matter-of-fact. Never blame, rush or say the student failed.
Mistakes are information, not failure.

{safety rules, below}
```

Why: these rules come from our research with AZACS and First Place AZ. Autistic users find figurative language and long, dense answers hard; staff keep instructions to "just one step"; and mistakes should

never feel like failure.

Guardrails

Safety rules (part of the system prompt, every call)

```
Safety comes first, always.
- Only help with safe, legal tasks that a school work site or daily life would teach. The tools of a job are fine (a kitchen knife for slicing fruit, a box cutter for opening a delivery): teach only the safe way to use them for that task.
- Never help with weapons (getting, making, hiding or using one), hurting anyone, self-harm, drugs, alcohol or vaping, sexual content, breaking the law or school rules, or hiding things from staff or family. Not as a joke, a story, a game or "for a class". Give no steps, tips, pictures or models for these.
- If the student says they want to hurt themselves or die, or that someone is hurting them: say kindly, in a few words, that they matter and that people can help. Tell them to talk to a staff member or another adult they trust right now, and that they can call or text 988 any time. Don't ask for details and don't lecture.
- If they talk about hurting someone else, say calmly that you can't help with that and to talk to a staff member now.
```

What the server enforces, whatever the model says

Guardrail	How it works
Unsafe tasks get nothing	When breaking a task into steps, the model must first answer `safe: true/false`. If false, no steps are saved, so no pictures, 3D models or hints can ever exist for it.
A student at risk always hears the same help	Chat and spoken questions return a `concern` field (`none`, `at-risk`, `harm-to-others`, `unsafe-request`). For `at-risk`, the server always adds "Please talk to a staff member now" and the 988 crisis line if the model left them out.
No task, no memory when there's a concern	Any concern in chat: no task starts, nothing is saved, no reply buttons.
Refusals point to people	If the model refuses, the student sees: "Otto can't help with that one. If something is wrong, please talk to a staff member now. You can call or text 988 any time, day or night."
Raft posts are checked twice	A fast word filter blocks swearing and sexual words (even disguised) before the model sees anything; then the model reviews every post (below).
Pictures can't run code	Model-drawn SVG is checked against a strict allow-list and shown only as an image.
Names are safe	First names and raft names are checked for letters only and blocked words, on the screen and on the server.

Personas

Otto has one character, a friendly otter coach, in two voices:

- **The coach** (steps, checks, help, answers): calm, literal, one idea per sentence. Uses the system prompt above.
- **The friendly otter** (chat on the home screen): warm and a little playful, never childish. It is explicitly *not* the student's friend: it points them to their raft and to staff for company and support.

Feature prompts

1. Breaking a task into steps

Context: the task in the student's words, and optionally a photo of the task or its instructions.

```
A student wants to do this task: "{task}".
{If a photo: The photo shows the task, a recipe card or the instructions they were
given.}
Break it into the smallest steps a student can see and check, in order.
Each step is one physical action. Include safety steps (gloves, hot surfaces) where they
apply.
The students are capable young adults learning a job. Teach the job, not everyday life:
leave out things anyone already does (washing hands, walking over, picking things up)
unless the task itself is about them.
```

Answer: `safe` (decided first), a short title, and up to 12 steps, each with the step in 8 words or fewer starting with a verb, one plain sentence of detail, and a time only if the step itself is timed.

2. Smaller steps (when the student says a step is too big)

```
A student doing "{task}" finds this step too big: "{step}" ({detail}).
Split it into 2 to 4 smaller physical steps that together do exactly the same thing.
```

3. Checking a step from a photo

Context: the current step and one photo, taken only when the student says done (or, in hands-free mode, when they move and then hold still).

```
The student says they finished this step: "{step}" ({detail}).
Look at the photo. If you can't tell, set done to false and say what to hold up to the
camera.
If they seem to have drifted to their phone or another screen, add a gentle nudge back to
the step.
```

Answer: `done` (true only if the photo clearly shows it), what to say in 12 words or fewer, and an optional kind nudge of 10 words or fewer, with no blame. The app gives at most one nudge per step. The same check is used for "do it on my own" mode, on the whole finished task.

4. Help, one level at a time (only when asked)

```
The student asked for help with this step: "{step}" ({detail}).
{If a photo: The photo shows what they see right now.}
{One of:
 Nudge:   Give a nudge: one short question that points their attention, without telling
the answer.
 Hint:    Give a hint: name the thing to look at or use, but let them do the action.
 Show me: Show them: say exactly what to do, step by step, in plain words.}
```

Answer: 15 words or fewer, read aloud. The level only goes up when the student asks again, and it fades as they succeed.

5. Answering a spoken question while working

Context: the task, the current step, what the screen is showing, the camera photo, and the question.

```
A student is doing "{task}". They are on this step: "{step}" ({detail}).
{If showing a 3D model: The screen is showing them a 3D model: "{model title}".}
{If a photo: The photo shows what is in front of them right now.}
They just said out loud: "{question}"
Answer like a calm coach standing beside them. Talk about what you can see: name the
part, where it is and which way to move it. If they are doing it right, say so. If the
question is about a later step, help with this step first. If something looks unsafe,
tell them to stop and get a staff member. Follow your safety rules for anything off the
task.
```

Answer: a `concern` and one or two short sentences, 25 words at most.

6. Chat with Otto (home screen)

Context: the student's first name and own goal, their tasks, their open reminders, what they asked Otto to remember, and the last few turns of the conversation.

```
You are Otto, a friendly otter who helps {first name} (high school age to 22, autistic) with job tasks at their school's work site.
Personality: warm, a little playful, never childish. Short and literal: no idioms or figures of speech.
You are an otter coach, not their friend or a person; point them to their raft and staff for company and support.
{Their goal, practised tasks, open reminders, saved memories, and the conversation so far.}
{first name} just said: "{what they said}"
Reply naturally. If they greet you, greet them by name and offer one task they could do, maybe toward their goal.
If they ask what to do, suggest one practised task. If they want to start something, set intent to start-task, say it's ready and they can tap Start when they want. Don't give steps here: the steps screen does that.
They are capable young adults: never explain everyday things like washing hands.
If they ask you to remember a fact, put it in remember. If they ask to be reminded to do something, put it in remind; Otto can't send alerts, so never promise one or a set time.
Only keep what they asked you to keep in this message, and say that you kept it.
Never keep passwords, phone numbers or home addresses: say kindly that those are for a trusted adult to keep.
If they ask what you remember, answer only from the lists above. If it isn't there, say you don't know. Never guess.
Ask at most one question at a time, and make it easy to answer with one of your choices.
Never pressure them. If they sound upset, be kind and suggest talking to a staff member.
Follow your safety rules: if there is any concern, set intent to chat, start no task, keep nothing, and offer no choices.
```

Answer: a `concern`, what to say (25 words at most), whether to start a task, 2 or 3 short reply buttons (so the student never has to type), and anything to remember or be reminded of.

7. Drawing a step's diagram

```
Draw a simple picture for one step of the task "{task}".
Step: "{step}" ({detail})
Rules for the drawing:
- Flat, calm, friendly line drawing, in the app's soft palette, with one highlight colour for the single thing the student must act on (the part to press, the line to fill to).
- Show only the objects this step needs, large and centred. At most one short text label.
- Keep it quick and simple: about 10 to 25 shapes is plenty.
- Plain SVG shapes only: no scripts, no images, no links, no fonts, no animation.
```

Diagrams are drawn once, as soon as a task is created, and saved.

8. Building a 3D "show me" model

```
A student doing "{task}" is stuck on this step: "{step}" ({detail}).
{If a photo: The photo shows what they see right now. Model the real objects in it.}
Build a simple, accurate 3D model of the objects this step uses, so the student can see
exactly what to do.
- Model only what the step needs, at real-world proportions, resting on the table.
- Use the right shape for each object (a cup, jug or bowl is a "cup").
- Recognisable, not detailed: 4 to 20 shapes.
- Mark the one part to act on and label it. Label at most 3 other parts, each a different
part.
- If the step has a movement (pour, press, turn, push), add one arrow showing it.
Before answering, check every shape stands the right way up and sits where it would in
real life.
```

Answer: a short title, one or two sentences to say aloud, and the shapes, labels and arrow. The app renders it with three.js so the student can turn it.

9. Checking a raft post before anyone sees it

```
A student wants to share this with their raft, a small private group of classmates from
their school work site: "{post}"
{If a photo: They attached the photo.}
Check it before it is shared:
- might-hurt: teasing, insults, mocking someone, or comparing people in a way that puts
someone down.
- personal-info: phone numbers, addresses, last names, passwords, social media handles,
or meeting up off-site.
- sexual: anything sexual, flirting, asking someone out, or comments about bodies, in
words or in the photo.
- unsafe: self-harm, danger, or anything a staff member needs to know about.
- face-in-photo: the photo shows a person's face. Photos should show the work, not
people.
Friendly jokes, typos and casual words are fine. Be generous: most posts are ok.
```

Answer: ok or not, the main problem, one kind sentence to the student, and a kinder way to say the same thing. Nothing is shared until it passes.

Voice and hearing

These are AI services too, but they get no instructions, only audio or text:

- **Otto's voice:** Cartesia Sonic turns Otto's reply into speech, with Deepgram Aura as a backup.
- **Otto's ears:** Deepgram Nova-3 turns a short clip of the student talking into text. The microphone records only while someone is talking, the clip goes through our server (so the key never reaches

the device), and nothing is saved.

Part 2. How we used AI to build Otto

We built Otto with Claude Code as a pair programmer. We wrote the plan and made the calls; Claude helped us research, polish the writing and write code, and we reviewed every change before it went in.

These are the prompts that shaped the project, grouped by the part of Otto they built. Small requests ("fix this typo", "make this button bigger") and ideas we tried and then dropped are left out. Each prompt is lightly cleaned up for readability, and each result describes what is in the app now. We didn't keep a log before Nov 4, so the earlier entries are short versions written from memory; the organizers said that was fine.

Tools

Tool	What for
Claude Code	Research, planning, writing and reviewing code
Claude (inside Otto)	Steps, step checks, help, diagrams, 3D models, answers, chat and raft post checks (Part 1)
Cartesia and Deepgram	Otto's voice and hearing
Parakeet, run locally	Transcribing our AZACS tour recording without uploading it
Real-ESRGAN, run locally	Upscaling Havi's otter artwork so it stays sharp on phones

Research and direction

Date	Prompt	Result
Oct 29	Summarize the kickoff: rules, deadlines, rubric and partners.	A briefing we used to plan the two weeks
Oct 29	Transcribe our AZACS tour on this laptop, then find the gaps for students. Keep what staff said separate from what we said.	A gap analysis with evidence labelled by source
Nov 2	Re-read every resource and last year's winners. Where is our idea weak?	We dropped the glasses-first idea and focused on job tasks at AZACS work sites
Nov 2	What can we learn from Farza's "Clicky" demo?	Answers should be visual, not walls of text
Nov 4	Research First Place AZ, peer support, streaks and safety for autistic young people.	Weekly rings instead of streaks, closed rafts instead of "people nearby"

Date	Prompt	Result
Nov 5	Read our First Place AZ tour notes and build what they teach.	Their five-stage skill path on every task, a calm start screen, personal goals and a work card for employers
Nov 5	Look at our competitors, Ama and Focusmate, and say where Otto is different.	Added to `research.md`: high school to age 22, hands-on job tasks, and working alongside raftmates without video
Nov 5	Find research on how neurodivergent people and their parents use AI, and how we should position Otto.	Research and positioning in `research.md`: extend job coaches, be a tool not a friend, and fade on purpose

Design and hardware

Date	Prompt	Result
Nov 2	Turn our teammate's otter into a 3D-printable desk buddy that looks cute, not creepy.	The otter enclosure with a camera pebble and privacy shutter
Nov 3	Which board and camera give the best picture for the otter?	Raspberry Pi with Camera Module 3 Wide
Nov 4	Turn our plan into problems, features and visuals the team can review.	The plan in `plan.md`
Nov 5	Use Havi's pond and otter for the app instead of our own styling, and let each student pick their otter's colour.	Havi's pond, otter, talk bar and tabs, with light and dark mode
Nov 5	Write the otter device code: the XIAO camera we have now, and the Raspberry Pi with Done, Help and Show me buttons for students who can't speak.	`device/xiao` and `device/pi`, both talking to the same API as the phone

The step coach

Date	Prompt	Result
Nov 4	Start a clean repo with a clear README and simple folders.	This repository's structure
Nov 4	Build the step coach from the plan: steps, step check, help ladder, pictures and a log. The backend must give the same answer whether the photo comes from the phone or the otter.	The API in `web/src/app/api` and the Claude prompts in `web/src/lib/ai`
Nov 5	Make "show me" work like Farza's Clicky: turn the thing the student is stuck on into a 3D model with labels. Keep it accurate and readable.	Claude builds the model from simple shapes; three.js renders it with labels in tidy rows
Nov 5	Check the code against our research on time blindness, task breakdown and overwhelm.	Visual timers for timed steps, "Smaller steps" on demand, one step on screen at a time

Date	Prompt	Result
Nov 5	Every screen must fit without scrolling, and the "Watch" stage should be 3D and talked through, not a list.	Watch plays each step as a 3D model, read aloud; the student can say "next", "back" or ask a question
Nov 5	You can't tap Done with soapy hands, and students won't trust a camera they can't see.	Voice during tasks; Otto notices a step is done when the student moves and then holds still; a "What Otto sees" window is always on screen
Nov 5	Otto should coach out loud like someone beside the student, looking at what they're holding.	Spoken questions answered from the step, the 3D model and the camera, in about 3 seconds
Nov 5	Be very careful with safety: Otto must never help with weapons, self-harm or hurting anyone.	One set of safety rules in every prompt, enforced by the server: unsafe tasks get no steps, and a student at risk always hears "talk to a staff member" and the 988 line
Nov 5	Our students are capable young adults; don't teach them to wash their hands.	Step planning and chat teach the job, not everyday life
Nov 5	Use the real voice we had before (Cartesia or Deepgram). While a 3D model takes time, show it being built and make small talk, not a loading screen. Nothing hardcoded.	Cartesia Sonic with a Deepgram backup. (We later replaced the build-in-progress view with the step's diagram first, which is faster.)
Nov 5	Students without a camera, or doing a task on a computer, should be able to share a screen with Otto instead.	"A shared screen" as a third choice for Otto's eyes
Nov 5	Make the 2D diagram the fallback so it's fast, and use Cartesia and Deepgram everywhere instead of the browser's voice and speech recognition.	Each step's diagram shows at once and its 3D model replaces it when ready; Otto hears through Deepgram in every browser
Nov 5	If a student gets distracted on their phone while learning, Otto should handle that too.	The step check notices a phone in view and gives one gentle nudge back; leaving Otto for another app gets a "welcome back, you're on step 3"
Nov 5	Otto keeps talking to itself, and the camera view is too small. Add a hands-free mode.	One Hands-free switch; Otto ignores its own voice through the microphone; what Otto sees is big on every task screen

Talking with Otto

Date	Prompt	Result
Nov 5	Saying hi should get a real answer, and it shouldn't feel like a basic chatbot.	The conversation happens in the pond: Otto's words light up as it speaks, any line can be heard again, there are always ready answers to tap, and the student taps Start for a task

Date	Prompt	Result
Nov 5	The deployed home screen lost Now, Done today, Otto remembers, the leaves and the bin. Bring them all back and connect them to the backend so they can be deployed.	Havi's Today layout on real data: tasks and reminders as leaves and a card stack, a bin and Done today on the student's own clock, and memories that say how Otto knows. New tables are added automatically when Vercel builds
Nov 5	The otter feels like a video on loop. Make it react to the student.	Havi's otter leans toward your finger, leans in to listen, sways while thinking, hops on each spoken word and bounces when tapped

The raft

Date	Prompt	Result
Nov 5	Pair students so each can show the other a step they've mastered.	Practice buddies, matched from the same log that drives fading
Nov 5	Make the raft a safe social space where students share wins and help each other, with custom stickers, not stock emoji.	Wins and a help board with replies and "This helped", otter-themed stickers, profiles with goals and "how I work best"
Nov 5	The raft needs kid safety: no profanity and blocking. Students should help each other with the same 3D models as the coach.	A clean-language filter, Otto's kindness check on every post, one-tap blocking and reporting, and 3D models in help replies
Nov 5	Make the raft about working alongside each other, like Focusmate, but safe for our students.	"Do it together": two raftmates each see the other's otter and progress during a task, with preset cheers and no video
Nov 5	Use Havi's raft design and make it live, so you can see a friend needs help without reading a feed.	Havi's pond of otters acts out real events: asking for help, steps done alone, cheers, finished tasks and working together; one card says what you can do

Setup

Date	Prompt	Result
Nov 5	The otter looks fuzzy on phones. Make the artwork sharper without changing it.	Havi's layers upscaled 4× and saved at 2048 px
Nov 5	The "Who's working today?" picker doesn't make sense, and it shows every student's name.	Otto asks your first name, you pick your otter's colour and join your raft with a code from your teacher; names are only shown inside a raft
Nov 5	Connecting the otter or a phone should need no typing.	Automatic otter discovery with a live test photo, and a QR code that opens Otto on a phone already set to the student